

ENEMY AI · 01

AI 의사결정

BG3 전투 모작 · 적 AI 턴 설계와 구현

Project B3 · 3인 팀 (개발자 3명) · 2026.03 (4주) · UE5.6

▶ 데모 영상

`</>` **GitHub** audidas/ProjectB3-code

프로젝트 개요

- BG3 전투를 모작한 턴제 전술 RPG를 UE5.6으로 구현한 3인 팀 프로젝트
- 적 AI 의사결정 시스템 담당 — 판단·실행·검증

01

의사결정 코어

Utility 스코어링 · 턴 플래너

02

실행 파이프라인

StateTree 턴 흐름 · GAS 어빌리티 연동 · EQS 위치 선정

03

검증 도구

AI 점수 디버거 · greedy 베이스라인 측정툴



목차

01	단일 행동 선택의 한계	04	07	DFS가 필요한 조건	10
02	턴 계획 접근	05	08	시스템 구조	11
03	행동 순서 탐색	06	09	실행 견고성	12
04	행동·타겟 스코어링	07	10	AI 디버거	13
05	시너지 스코어링	08	11	한계와 개선 방향	14
06	greedy 베이스라인 비교	09	12	맡은 작업	15

단일 행동 선택의 한계

구조 설계 과정 — 규칙 기반 → 점수 선택 → 턴 계획

- ① **규칙 기반(if-else)** — 상황 조합마다 분기 증가 · 행동 패턴이 읽힘 → 행동별 점수를 부여하여 높은 점수 선택(Utility)
- ② **높은 점수만 반복(greedy)** — 자원은 다 쓰지만 시너지를 구조적으로 고르기 힘들
- ③ → **순서까지 탐색하는 턴 계획 필요**

한 턴 자원 — 액션(AP) 1 · 보너스 액션(BA) 1 · 이동력(MP)

높은 점수 반복 — greedy

공격 · AP +6

→

밀치기 · BA +2

→

자원 소진

자원은 전부 사용 — 그러나 순서는 매 순간 1등 순
턴 점수 = $6 + 2 = 8$

턴 계획 — 순서 전체를 탐색

밀치기 · BA +2

→

이동 · MP

→

공격 · AP +6

같은 자원, 같은 행동 — 순서만 바꿔 시너지 확보
턴 점수 = $2 + 6 + 7(\text{장판 보너스}) = 15$

행동 하나가 아니라 한 턴을 계획

한 턴 행동 순서 전체의 누적 점수로 최고 순서를 탐색



Macro-action — 이동은 부속 비용

이동을 독립 행동이 아닌 각 행동의 부속 비용으로 처리 → 탐색 분기 수 축소. 구체 위치 선정은 EQS 담당.

DisplacedTargets — 밀린 상태 추적

밀치기 분기에서 밀린 타겟은 근접 후속 후보에서 자동 제외. 값 복사 컨텍스트라 백트래킹 시 자동 원복.

규칙을 넣지 않고, 점수와 탐색만으로 순서를 결정

행동 순서 탐색

행동 순열을 DFS로 탐색 — 상한 기반 가지치기(B&B)로 최선의 조합을 놓치지 않고 탐색량만 줄임

탐색 노드

노드 = 누적 행동 경로 + 잔여 자원 덱스 4 +
AP·BA 제한으로 통상 덱스 2~3에서 종료 분기
폭 = (타겟 × 어빌리티) + 광역 + 다중타겟 후보
— 사거리·LoS·자원으로 생성 단계에서 사전
필터

Branch & Bound

상한이 실제 도달값을 과소평가하지 않게 시너지
보너스(30)까지 선반영 → 최선의 조합 유지 ·
콤보 분기 조기 컷 방지

값 타입 백트래킹

분기마다 컨텍스트 구조체를 값 복사
→ 잔여 자원·가상 HP·밀린 타겟 자동 복원

점수 캐시

모든 점수는 DFS 전에 사전 계산
→ 탐색 내부 재계산 0

가지치기 — 예시 (현재 최고점 35일 때)

공격

상한 42 > 35 → 계속 탐색

이동 ×

상한 30 ≤ 35 → 하위 경로 전체 폐기

// 기대한 점수 상한이 최선 이하면 가지 전체 폐기

```
MaxRemaining = Min(AP+BA, MaxDepth-Depth);
UpperBound = Score + MaxSingle*MaxRemaining;
if (UpperBound <= BestScore) return;
```



한 턴에 이동 + 공격 — 적이 남은 이동력으로 접근한 뒤 같은 턴에 공격까지 연결
자원 배분을 탐색이 결정

행동·타겟 점수 — 명중 확률까지 반영

$$\text{ActionScore} = (\text{ExpectedDamage} \times \text{ThreatMult} \times \text{RoleMult} + \text{SituationalBonus}) \times \text{ArchetypeWeight}$$

기대피해

(ExpectedDamage)

단순 평균이 아니라 d20 명중·치명·세이브 확률로 계산한 기대값

위협도

(ThreatMult)

Lerp(0.5~2.0) · 턴 내 상대 정규화 — 절대값 곱은 적 수에 따라 정확도가 낮음

타겟 직군

(RoleMult)

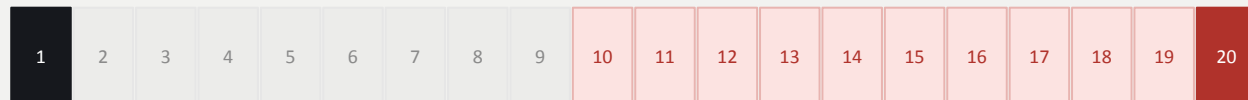
힐러 1.3 > 캐스터 1.2 > 원거리 1.1 > 근접 1.0 > 탱커 0.8

처치보너스 · 성향

(SituationalBonus · ArchetypeWeight)

처치는 오버킬 보정으로 딱 죽일 만큼만 인정 · 성향은 DataSet 가중치

기대피해 — d20 확률 축 (예: 명중 최소 눈금 10)



1~9 실패 · 1 = 자동 실패

10~19 명중 · 20 = 치명

기대피해 = $P(\text{명중}) \times \text{평균피해} + 1/20 \times \text{치명피해(주사위 2배)}$

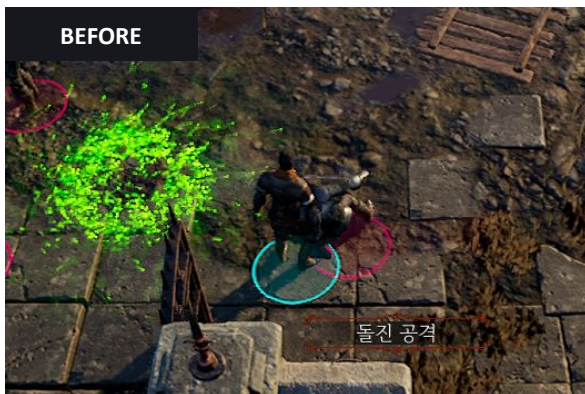
세이브 스킬은 성공 시에도 절반 피해로 모델링



약한적부터 공격

시너지 스코어링 · 공격 낭비 방지

BEFORE / AFTER — 장판 주변 확대



밀치기 전 — 플레이어가 장판 밖



밀친 뒤 — 위험 장판 위로 진입

[PBUtilityClearinghouse.cpp:3588, :3634](#)

위험 장판 시너지

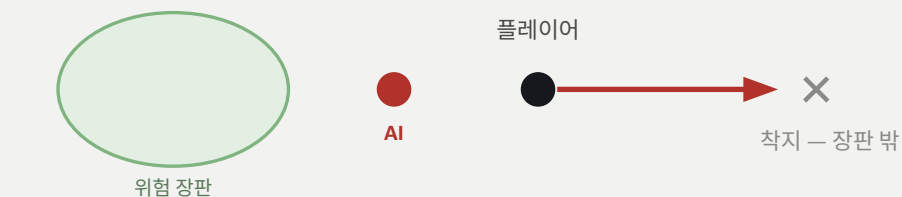
밀린 뒤 착지 위치를 예측 — 위험 장판이면 장판데미지 $\times 1.5$ 를 그 분기 점수에 가산

VirtualHP

공격마다 기대피해를 예측 차감, 사망 예측 타겟은 후속 후보에서 제외 · 값 복사로 분기별 독립

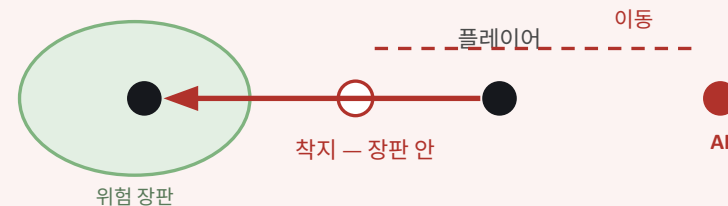
콤보 턴 점수 — 같은 전투 상황에서 두 방식이 각각 계획한 한 턴의 누적 점수. 아래 도형이 점수가 갈린 원리

greedy 계획 0.63



단일 스텝 선택은 “A를 해야 B의 가치가 열린다”를 구조적으로 보지 못함 — 2행동 시퀀스 탐색만 발견

DFS 계획 6.19



greedy 베이스라인 비교

- 채점을 공용 헬퍼로 추출 — DFS·greedy를 동일 기준으로 비교
- 로그 전용, 게임플레이 영향 0 — 매 턴 두 시퀀스를 병행 채점해 누적 기록
- 0행동 턴은 분모에서 제외

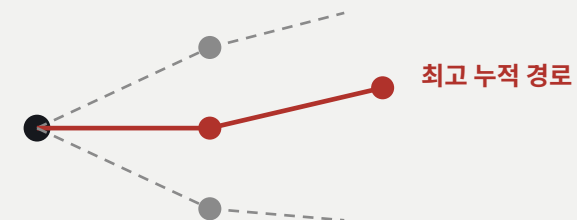
탐색 경로 비교

greedy



매 스텝 argmax 1개 — 한 줄 직진, 백트래킹 없음

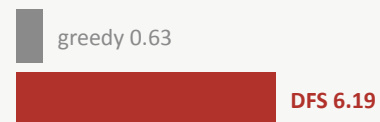
DFS



모든 후보 분기 후 백트래킹 — 누적 점수 최대 경로 선택

갈리는 순간 — 콤보 두 케이스

콤보 ①



콤보 ②



일반 전투 (17 의사결정 턴) — 평균 차이 +0.09, 턴 점수의 약 0.3% · DFS $\approx$ greedy

→ 앞 행동이 뒤를 받쳐주는 콤보에서만 DFS가 갈림

언제 DFS가 필요한가

가정 1

단순 Utility는 콤보 못 만든다 → 계획 필수

→ 일반 전투에서는 greedy도 동일한 결정에 도달 — 계획이 항상 필수는 아님

가정 2

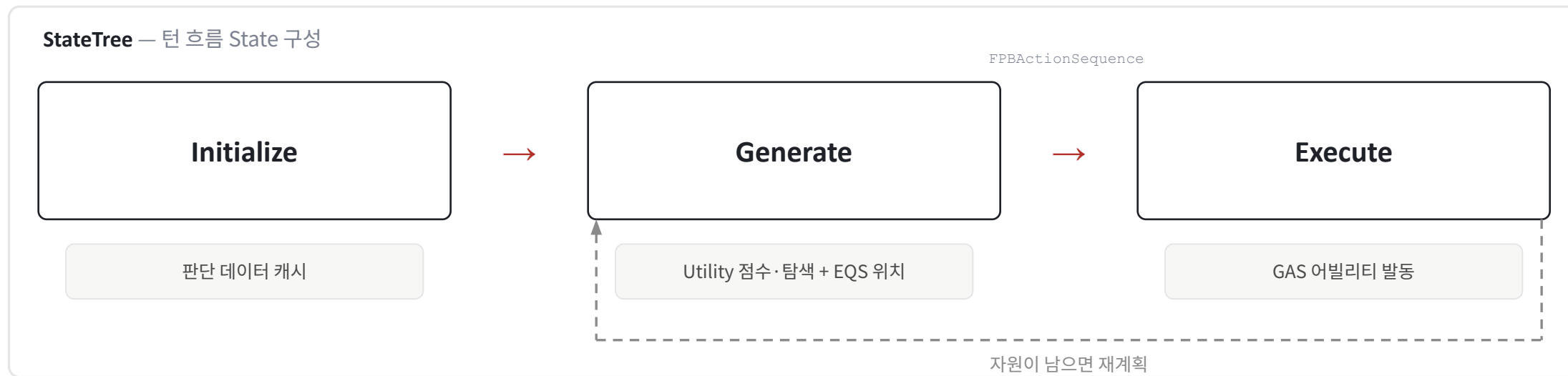
점수로 콤보가 나오니 greedy도 충분하다

→ greedy만으로는 시너지 효과를 내는 콤보를 고를 수 없기 때문에 틀림

측정 결과

DFS는 행동이 독립적이면 greedy와 차이 없고, 앞 행동이 뒤를 받쳐줄 때만 갈림

시스템 구조 — 판단·흐름·위치 분리



결과 — 후보 카테고리 7종이 같은 패턴으로 처리 · 신규 어빌리티는 후보 등록만으로 탐색에 포함 (이동은 후보가 아닌 각 행동의 부속 비용)

공격

힐

버프

디버프

제어

광역

다중타겟

실행 견고성 — 계획과 실행의 간극

계획 시점 $\neq$ 실행 시점 — 시퀀스 실행 중 전투 상태가 변함 $\rightarrow$ 실행 직전 재검증 + 비동기 콜백 체이닝으로 대응



GAS 어빌리티 종료 콜백으로 비동기 체이닝 · 콜백 유실 대비 3초 타임아웃

실행 직전 검증 — 3중 안전장치

01

LoS 재검증

계획 시점의 시야가 실행 시점에도 유효한지 발동 직전 재확인

02

타겟 사망 처리

타겟이 이미 죽었으면 후속 행동 무효화 · 소모 예정
자원은 보존 — Generate 재진입 시 재계획에 사용, 턴을
버리지 않음

03

비용 강제 차감

비용 처리 누락에 대비해 실행 시점에 자원을 강제 차감

결과 — 개발 후반 신규 어빌리티 추가에도 크래시 없이 적응

AI 디버거 — 점수 출력

- 도입 이유 — 점수 기반 AI는 행동만으로 판단 근거 파악 불가 → 점수 출력으로 튜닝·버그 추적 확보
- 출력 항목 — 후보별 기대피해·위협배수·액션·총점 + 생성 시퀀스 + 실행 상태

게임 오버레이를 읽기 쉽게 재배치

```

=== AI Utility Debugger: BP_Player_Faelis_C_0 ===
[Cache] Targets: 3  Allies: 6  MaxMP: 900 cm
[Archetype] Atk=1.5 Heal=0.1 Buff=0.1 Ctrl=0.1 ◀ ②

--- Attack Scores (Top 3) ---
Zadikal : ExpDmg=3.5 TM=2.40 Action=12.4 Total=12.50 ◀ ①
Faelis  : ExpDmg=3.5 TM=1.79 Action=9.3  Total=9.25
Varain  : ExpDmg=3.5 TM=1.25 Action=6.5  Total=6.66

--- Generated Sequence ---
>>[0] Move    -> (-7852, -4661, 15)  Zadikal    ◀ ③
[1] Attack    -> Zadikal
[2] Control   -> Zadikal [Displacement]
TotalScore: 12.58  Ready: YES

```

디버거로 확인하는 것들

① 타겟 선정 근거

TM = 위협 배수. Zadikal이 2.40으로 최고 → 이 턴 공격 1순위로 뽑힌 근거를 숫자로 확인

② 아키타입 가중 검증

Atk=1.5 = 공격형 성향 가중치. 공격 점수에 실제로 ×1.5가 반영됐는지 총점으로 검증

③ 시퀀스·실행 추적

Generated Sequence = 계획된 행동 순서 · Ready: YES = 실행 준비 완료 · 실행 중 진행 상태 추적

한계 8가지

한계와 개선 방향

	한계	근거	상태
L1	효과 미검증 (DFS vs greedy)	복잡하니 좋다 — 데이터 부재	● 완료 · greedy 측정툴 구현
L2	성급한 단정 (planning 필수)	일반 전투는 greedy로 충분 — 측정으로 확인	● 완료 · 논지 재구성
L3	EV 모델 단위 불일치	VirtualHP는 기대값, 처치 판정은 raw 평균	○ 예정 · 단위 통일
L4	추정-실행 불일치	계획은 직선거리 × 1.3 추정, 실제 위치는 EQS 결정	○ 예정 · 측정·완화
L5	죽은 코드	고지대 평가가 계산만 되고 스코어 미연결	○ 예정 · 살리거나 제거
L6	비대한 클래스 (4000줄)	단일 함수 540줄 — 스코어링·탐색·위치 책임 혼재	○ 예정 · 책임 분리
L7	중복 코드 (이동비용 4중)	동일 블록 4회 반복 — 채점 헬퍼 추출이 첫 사례	● 완료 · 헬퍼 추출 착수
L8	하드코딩된 보정값	위협 Lerp·처치 상수 하드코딩 · response curve 0개	○ 예정 · 데이터화

맡은 작업

01

스코어링

02

행동 순서 탐색

03

StateTree Task

04

EQS 연동

05

AI 디버거

회고

밀치기 → 장판 시너지는 구현했으나, 화약통·시너지 스킬까지 엮지 못한 점이 아쉬움 화약통이 터질 때의 이득까지 AI가 읽게 만드는 작업은 기회가 되면 이어서 붙여볼 계획

THANK YOU

감사합니다

최승현

▶ 데모 영상

`</>` **GitHub** audidas/ProjectB3-code